

Control 2 - Técnicas Computacionales para Radio Interferometría Astronómica

Prof. Miguel Cárcamo

2 de diciembre de 2025

Puntaje total: 6.0 puntos

Instrucciones: Responda todas las preguntas en un Jupyter notebook. Asegúrese de ejecutar todas las celdas y verificar que su código funciona correctamente.

1. Pregunta 1: Dask (1.5 puntos)

En radio interferometría, frecuentemente necesitamos procesar grandes volúmenes de datos de visibilidades o múltiples épocas de observación. Dask permite manejar estos datos de manera eficiente mediante computación distribuida.

a) (0.5 puntos) Simule un conjunto de visibilidades sintéticas griddeadas en el espacio uv . Las visibilidades griddeadas tienen forma de matriz $N_u \times N_v \times T$, donde N_u y N_v son las dimensiones de la grilla uv y T es el número de tiempos de integración. Cree un array de Dask de tamaño $1000 \times 1000 \times 100$ (representando visibilidades griddeadas en una grilla uv de 1000×1000 píxeles para 100 tiempos de integración) particionado en bloques de $200 \times 200 \times 20$. Calcule el promedio temporal de las visibilidades usando `compute()`.

b) (1.0 punto) Cree 50 imágenes sintéticas de síntesis de apertura (arrays numpy complejos de tamaño 4096×4096), donde cada imagen corresponde a una época de observación diferente. Implemente una función que use Dask para procesar en paralelo este conjunto de imágenes. La función debe calcular el promedio temporal de las imágenes y aplicar una transformación logarítmica al módulo del resultado ($\log(1 + |\text{promedio}|)$). Retorne el resultado como un array de Dask y muestre cómo ejecutar el cómputo eficientemente.

2. Pregunta 2: Numba CPU y CUDA (2.0 puntos)

En radio interferometría, el cálculo de funciones de correlación cruzada entre visibilidades griddeadas requiere operaciones computacionalmente intensivas sobre grandes volúmenes de datos, especialmente cuando se analizan variaciones temporales o entre frecuencias.

a) (1.0 punto) En radio interferometría, la función de correlación cruzada entre dos conjuntos de visibilidades griddeadas es útil para analizar variabilidad temporal o entre frecuencias. Implemente una función usando Numba CPU (`@numba.jit`) que calcule la correlación cruzada 2D entre dos matrices de visibilidades griddeadas complejas. Dadas dos matrices de visibilidades V_1 y V_2 de tamaño $N \times N$ (donde cada elemento es un número complejo), la función debe calcular la correlación cruzada con condiciones de frontera periódicas:

$$C[i, j] = \sum_{k=0}^{N-1} \sum_{l=0}^{N-1} V_1[k, l] \cdot \overline{V_2[(k+i) \bmod N, (l+j) \bmod N]} \quad (1)$$

donde $\bar{z}$ denota el conjugado complejo y el módulo asegura condiciones de frontera periódicas. La función debe retornar una matriz compleja de tamaño $N \times N$ y estar optimizada con Numba. Para probar la función, simule dos conjuntos de visibilidades sintéticas griddeadas de tamaño $N \times N$.

b) (1.0 punto) Implemente la misma función pero usando Numba CUDA (`@cuda.jit`) para ejecutarla en GPU. Compare el tiempo de ejecución entre ambas versiones usando matrices de visibilidades sintéticas griddeadas de tamaño 2048×2048 . Discuta las ventajas de usar GPU para este tipo de cálculos en el contexto de procesamiento de datos de interferometría.

3. Pregunta 3: CuPy (1.5 puntos)

El procesamiento de imágenes de síntesis de apertura en radio interferometría requiere transformadas de Fourier y operaciones matriciales sobre grandes volúmenes de datos. CuPy permite acelerar estos cálculos utilizando GPUs.

a) (0.5 puntos) En el contexto de procesamiento de visibilidades griddeadas en radio interferometría, frecuentemente necesitamos realizar operaciones matriciales sobre grandes volúmenes de datos. Simule dos conjuntos de visibilidades sintéticas griddeadas (arrays complejos de CuPy de tamaño 4096×4096) y calcule su producto matricial usando CuPy. Esta operación puede ser útil en algoritmos de deconvolución o en el procesamiento de datos multi-frecuencia. Muestre el tiempo de ejecución.

b) (1.0 punto) Cree una imagen sintética de síntesis de apertura (array complejo de tamaño 8192×8192). Implemente una función que use CuPy para aplicar una transformada de Fourier 2D a esta imagen, luego multiplique el resultado en el espacio de frecuencias por un filtro de ventana gaussiana (para suavizar el ruido) y finalmente aplique la transformada inversa para obtener la imagen filtrada. Compare el tiempo de ejecución con la versión equivalente usando NumPy y discuta la importancia de la aceleración por GPU para el procesamiento de grandes imágenes de radio interferometría.

4. Pregunta 4: pytest (1.0 punto)

En el procesamiento de datos de radio interferometría, es crucial validar que las funciones de procesamiento funcionen correctamente con diferentes tipos y tamaños de datos. pytest permite automatizar estas verificaciones.

a) (0.5 puntos) Escriba una función que calcule el ruido RMS (root mean square) de un conjunto de visibilidades complejas (array numpy complejo). El ruido RMS se calcula como el promedio de los RMS de las partes real e imaginaria: $\text{RMS} = 0,5 \times (\text{RMS}(\text{real}) + \text{RMS}(\text{imag}))$, donde RMS corresponde a la desviación estándar. Cree al menos 3 tests usando pytest que verifiquen diferentes casos: simule visibilidades sintéticas no griddeadas con ruido gaussiano, visibilidades con un solo valor, y visibilidades vacías (debe lanzar una excepción apropiada).

b) (0.5 puntos) Cree un test parametrizado usando `@pytest.mark.parametrize` que pruebe la función de ruido RMS. Para cada test, simule visibilidades sintéticas no griddeadas con diferentes números de elementos (100, 1000, 10000) y diferentes niveles de ruido. Verifique que el ruido RMS calculado esté dentro de un rango esperado dado el nivel de ruido simulado.